



Coset Ensemble Decoder for Quantum Error Correction with Algorithm–Hardware Co-Design

Shuang Liang*, Jubo Xu*, Giulio Bassanino*, Qianzhou Wang*, Yidong Zhou†, Yuncheng Lu*, Zhiwen Mo*, Paul Kelly*, Bo Yuan†, Wayne Luk*, and Hongxiang Fan*

*Imperial College London, United Kingdom †Rutgers University, United States
{shuang.liang, jubo.xu20, p.kelly, w.luk, hongxiang.fan}@imperial.ac.uk

Abstract—Reliable large-scale quantum computation relies on fault-tolerant architectures, where quantum error correction (QEC) continuously extracts and decodes error syndromes in real time. A critical component in QEC is the decoder, a classical subsystem that must simultaneously deliver high logical accuracy and ultra-low latency. This paper presents a novel algorithm–hardware co-design that improves the accuracy–latency trade-off over existing approaches such as vanilla Minimum-Weight Perfect Matching (MWPM) and Union-Find (UF) decoders. At the algorithmic level, we introduce coset ensemble decoding, which improves UF decoding by explicitly exploiting logically equivalent cosets. Our method performs ensemble forest exploration to generate multiple coset-consistent candidates and aggregates them to approximate coset-level maximum-likelihood decoding. We further reduce computational and memory complexity via reverse-order elimination and lossless graph compression, without sacrificing accuracy. At the hardware level, we design a domain-specific architecture that temporally reuses resources, avoiding the code-distance-proportional resource growth in prior spatial architectures. Several optimizations, such as multi-bank memory hashing and hierarchical ID mapping, are proposed to mitigate pipeline stalls and memory conflicts under highly concurrent access patterns. Under a circuit-level depolarizing noise model, our co-design approach achieves a better accuracy–latency trade-off than prior MWPM- and UF-based decoders, while reducing FPGA LUT consumption by up to 8.2 times compared with reported UF-based decoder resources. The tunable candidate number further exposes a flexible design knob, enabling users to tailor decoding performance to the requirements of different fault-tolerant workloads. Our implementation is publicly available at <https://github.com/IMSeonL/coset-ensemble-decoder>.

Index Terms—Quantum error correction, coset ensemble decoding, Union-Find decoder, algorithm–hardware co-design, FPGA acceleration

I. INTRODUCTION

Recent advances in quantum computing have led to rapid growth in the number of physical qubits. However, existing quantum devices continue to suffer from gate errors, crosstalk, leakage, and limited coherence times [1], all of which hinder the execution of practical quantum circuits at scale. Achieving practical, large-scale quantum computation therefore requires fault-tolerant quantum computing (FTQC), which relies on quantum error correction (QEC) to encode logical qubits across multiple physical qubits. QEC continuously extracts error syndromes and corrects errors during runtime [2], suppressing the logical error rate below a desired threshold as guaranteed by the threshold theorem [3], [4].

At the core of QEC is the decoder, a classical subsystem that interprets error syndromes and generates corrections in real time. Designing such decoders is challenging due to the dual demands of high decoding accuracy and ultra-low latency. As quantum processors operate at extremely high speeds, decoders are typically required to process syndromes within less than 1 μ s in superconducting circuits, posing significant challenges for both decoding algorithm design and hardware implementation. Among various QEC codes, the surface code has emerged as a leading candidate due to its high threshold and local stabilizer structure [5]. Within the surface code framework, two prominent decoding strategies have been extensively adopted: Minimum-Weight Perfect Matching (MWPM) [6] and Union-Find (UF) [7]. MWPM achieves high accuracy by solving an optimal matching problem over the syndrome graph, but its high computational cost and inherent sequential nature of computation limit its processing speed. In contrast, the UF decoder achieves lower latency by clustering defects in parallel yet sacrifices accuracy under complex noise conditions.

To address these challenges, this work proposes an algorithm–hardware co-design for accurate, low-latency, and flexible QEC decoding. At the algorithmic level, we introduce coset ensemble decoding that enhances traditional UF decoding by considering logical-equivalent cosets, the group of physical errors with the same syndrome and logical effect. At the hardware level, we develop a customized architecture tailored to the proposed algorithm. In contrast to prior work [8], [9], which relies on spatial architectures whose hardware resources grow with code distance, our hardware features high generality and efficiency through temporal reuse and targeted pipeline optimizations. Together, these choices improve the accuracy–resource trade-off while enabling tunable decoding accuracy for different deployment needs.

This paper makes the following main contributions. The first improves decoding accuracy, while the latter two reduce decoding cost and latency, and improve hardware efficiency.

- A novel *coset ensemble decoding algorithm* that leverages logically equivalent cosets to approximate coset-level maximum-likelihood decoding. This is achieved by ensemble forest exploration, which effectively improves the algorithmic performance.
- Several *algorithmic optimizations*, including reverse-order

elimination and lossless graph compression, together with an optimality analysis of the proposed algorithm, to further reduce computational complexity.

- A *domain-specific hardware architecture* tailored for coset ensemble decoding. It achieves high generality and efficiency through temporal reuse and targeted pipeline optimizations. Several hardware optimizations, such as multi-bank memory hashing and hierarchical ID mapping, are proposed to further improve hardware efficiency and reduce latency.

II. BACKGROUND AND MOTIVATION

A. Background

1) *Quantum Error Correction (QEC)*: QEC employs protocols to protect quantum information from decoherence and noise [10]. An $[[n, k, d]]$ QEC code encodes k logical qubits into n physical qubits ($n > k$), overcoming fundamental constraints, like the No-Cloning Theorem and projective measurements' destructive nature, through non-local encoding and indirect syndrome extraction using ancilla qubits. The code distance d , defined as the minimum number of physical qubits supporting an undetectable logical error, quantifies error resilience. The logical error rate can be exponentially suppressed by increasing d if the physical error rate is below a threshold.

2) *Stabilizer Code*: An $[[n, k]]$ stabilizer code has stabilizer group $\mathcal{S} \subset \mathcal{P}_n$ generated by $m = n - k$ independent commuting Pauli operators. One may choose a canonical generating set $\{S_g, T_g, \bar{X}_j, \bar{Z}_j\}$ [11], where S_g are stabilizer generators, $\bar{X}_j, \bar{Z}_j$ are logical Paulis for the j -th logical qubit, and T_g are *pure errors* that generate the pure error group $\mathcal{T}$ and satisfy $\{T_g, S_g\} = 0$ and $[T_g, S_h] = 0$ for $h \neq g$. Moreover, there exists an n -qubit Clifford unitary (an encoding circuit) U such that

$$S_g = UZ_gU^\dagger, \quad T_g = UX_gU^\dagger, \quad g = 1, \dots, m, \quad (1)$$

$$\bar{Z}_j = UZ_{m+j}U^\dagger, \quad \bar{X}_j = UX_{m+j}U^\dagger, \quad j = 1, \dots, k, \quad (2)$$

where Z_g and X_g represent Pauli- Z and Pauli- X applied on the g -th qubit. The commutation relation between T_g and $S_{g'}$ is given by

$$S_g \cdot T_{g'} = UZ_gU^\dagger UX_{g'}U^\dagger = (-1)^{\delta_{g,g'}} T_{g'} \cdot S_g \quad (3)$$

The logical operators satisfy the Pauli commutation relation $\bar{X}_j \cdot \bar{Z}_{j'} = (-1)^{\delta_{j,j'}} \bar{Z}_{j'} \cdot \bar{X}_j$, and a code word from a stabilizer code space is a state of the form $U(|0\rangle^{\otimes m} \otimes |\psi\rangle)$ where $|\psi\rangle \in (\mathbb{C}^2)^{\otimes k}$ [11].

3) *Surface Code*: Surface code is a promising stabilizer code known for its high error threshold and compatibility with 2D lattice architectures with nearest-neighbor interactions [6]. As illustrated in Fig. 1, a single logical qubit is encoded in a $d \times d$ array of physical (data and ancilla) qubits. A key property is that a data qubit error anti-commutes with adjacent stabilizers of the opposite type: an X error flips two neighboring Z stabilizers, and a Z error flips X stabilizers. This results in non-trivial syndrome signals only at error

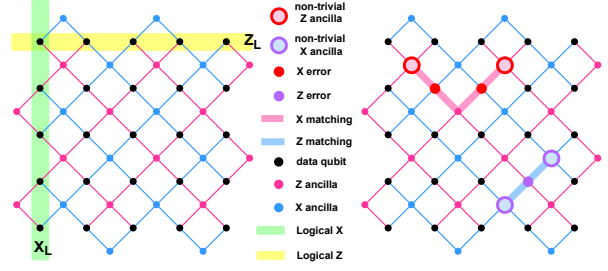


Fig. 1. Layout of surface code (left) and syndrome behavior with corresponding matching for X and Z errors on data qubits (right).

chain endpoints, crucial for decoding. To counter complex errors under circuit-level noise [12], decoding employs d syndrome rounds, XOR-ing consecutive syndrome outputs to form detectors for identifying measurement errors [5], isolating measurement errors from data qubit errors. Based on this, a 3D graph $G(\mathcal{V}, \mathcal{E})$, where vertices are detector events and edges are potential errors, can be constructed for decoding tasks [13].

4) *Degeneracy*: Stabilizer codes, like Classical Error Correction (CEC), have the property that multiple error patterns can produce the same syndrome measurement [14]. However, different QEC errors may be physically indistinguishable. In QEC, any error operator E can be decomposed into three components reflecting its stabilizer, pure error, and logical parts respectively as [11]:

$$E = s(E) \cdot t(s) \cdot l(E) = (s(E)l(E)) \cdot t(s) \quad (4)$$

where $s(E) \in \mathcal{S}$, $l(E) \in \mathcal{L}$, $s(E)l(E) \in \mathcal{N}(\mathcal{S})$. Pure error term $t(s)$ depends only on syndrome s : $t(s) = \prod_{g=1}^m T_g^{\frac{1-s_g}{2}}$, where $s_g \in \{+1, -1\}$ is the measured eigenvalue of the g -th syndrome. Thus, the same syndrome can correspond to distinct error patterns with same (differing only in $s(E)$) or different (differing in both $s(E)$ and $l(E)$) logical errors. Consider the first case, where errors differ only in $s(E)$. This happens if $E_1 = E_2 \cdot S$ with $S \in \mathcal{S}$. Under this condition, $E_1 |\psi\rangle = E_2 S |\psi\rangle = E_2 |\psi\rangle$, and the same correction R reverses both: $RE_2 |\psi\rangle = |\psi\rangle \Rightarrow RE_1 |\psi\rangle = RE_2 S |\psi\rangle = RE_2 |\psi\rangle = |\psi\rangle$. Such cases define indistinguishable degenerate errors, differing in physical error pattern but sharing both syndrome and logical effect, that can be grouped into the same logical-equivalent coset defined as $\{E|E = S_g t(s) L, \forall S_g \in \mathcal{S}\}$, which significantly impact decoding [15] (explained in Sec. II-A5).

5) *Optimal Decoding (Coset vs. Physical)*: Similar to CEC, it is straightforward to find the most probable physical error from syndrome s via Maximum-Likelihood (ML) decoding:

$$E^* = \operatorname{argmax}_{E \in \mathcal{P}_n} \{p(E|s)\} \quad (5)$$

However, degeneracy implies that the most likely physical error may not yield the most probable logical error, since multiple equally probable errors can have the same logical

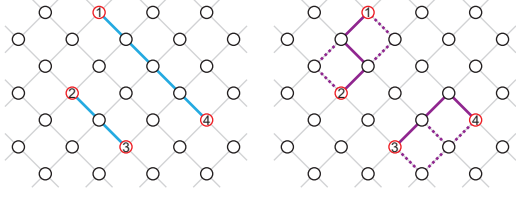


Fig. 2. Two matching examples: the blue (left) belongs to coset 1 with logical error L_b , the purple (right) to coset 2 with logical error L_p . Dashed lines indicate alternative error paths.

effect [16]. For example (just for illustration), as shown in Fig. 2, with non-trivial syndromes $\{1, 2, 3, 4\}$, error patterns can be matched as $\{1, 4\} \cup \{2, 3\}$ or $\{1, 2\} \cup \{3, 4\}$, each implying a different logical error. Both matchings contain 6 errors (edges) and therefore have the same weight, which means they are equally probable. However, the second matching includes 9 combinations with equivalent probability, so the total probability of the second logical error would be higher than that of the first. Since the two matchings belong to different logical cosets, and the second has higher probability, we choose an error pattern from the second coset—even though a pattern in the first also has maximal individual likelihood. Thus, rather than identifying the most likely physical error, optimal decoding should find the most likely logical coset [11]:

$$L^* = \underset{L}{\operatorname{argmax}} \{p(L|s)\}$$

$$p(L|s) \propto \sum_{E: l(E)=L} p(E) = \sum_{S \in S} p(E = St(s)L) \quad (6)$$

6) *Prior Decoding Algorithms:* The two main surface code decoders are Minimum-Weight Perfect Matching (MWPM) and Union-Find (UF). MWPM addresses $\operatorname{argmax}\{p(E|s)\}$ by solving the minimum-weight perfect matching problem on the decoding graph via the Blossom algorithm [8], which formulates the problem in a linear programming (LP) framework [17]. While this LP-based iterative procedure yields near-optimal decoding accuracy by maintaining *Blossoms*, it incurs substantial algorithmic and implementation complexity [18], as well as relatively large decoding latency for real-time scenarios. In contrast, UF, an approximation of MWPM, is a simplified and near-linear-time algorithm [7] that clusters syndromes by growing and merging them until all clusters have even parity, then resolves matches via spanning tree generation and peeling. While UF offers much lower latency due to its simplicity and high parallelism [19], [20], this comes at the cost of suboptimal accuracy compared to MWPM.

B. Motivation

Prior hardware decoders (i) neglect the impact of degeneracy [7], [13], and (ii) often rely on highly customized designs to achieve ultra-low latency, resulting in poor scalability [20]. Our work revisits both the decoding theory and the hardware architecture, leading to a coset-ensemble algorithm and a stall-resilient pipelined architecture.

1) *Algorithmic Challenge: Coset vs. Physical:* Prior decoders implicitly frame decoding as a physical ML problem: given a syndrome, they select the most likely physical error chain. In highly degenerate stabilizer codes, many distinct chains belong to the same logical coset, and the single most likely chain does not necessarily correspond to the most probable logical error. As a result, the decoder is optimized for recovering a particular localized configuration, while system-level reliability is determined by the logical error rate.

Insight and Our Approach. Given a syndrome, the ideal decision selects the coset with the highest posterior probability by summing over all compatible physical errors. This coset maximum-likelihood rule directly optimizes the logical error rate but may have exponential complexity in the code size [21].

Inspired by the above insight, we approximate the intractable coset ML issue with our *Ensemble Forest Exploration*. By using UF-equivalent *Clustering* to partition intractable cosets and independently injecting random priority during the forest construction, the decoder generates a set of candidate corrections that implicitly “vote” for different logical cosets. Aggregating these outcomes at the logical level yields an approximate ranking of cosets in polynomial time.

2) *Architectural Challenge: Scalability vs. Latency:* Grid-mapped hardware decoders achieve extremely low latency but suffer from poor scalability and efficiency. A more scalable Von Neumann-style organization naturally decomposes the decoder into two stages: a *pipelined clustering engine* that streams one vertex per cycle, followed by *post-clustering traversal modules* (e.g., spanning-tree construction and peeling in conventional UF) that run once clustering terminates. Profiling this two-stage baseline in Fig. 3 reveals that the dominant bottleneck lies inside the clustering pipeline: stalls in the clustering pipeline alone account for 48%–58% of the total decoding latency.

Insight and Our Approach. Targeting an efficient decoder, we observe that the vast majority of stalls arise from two dominant patterns of concurrent memory accesses during cluster growth and merging stages.

Guided by this observation, we co-design a conflict-aware multi-bank hashed memory system and a hierarchical ID mapping scheme to resolve these two classes of conflicts and keep the pipeline highly utilized.

III. COSET ENSEMBLE DECODER

This section presents three algorithmic contributions that together define our coset ensemble decoding procedure (Algorithm 1): *ensemble forest exploration* (Sec. III-A), *reverse-order elimination* (Sec. III-B), and *lossless graph compression* (Sec. III-C).

A. Ensemble Forest Exploration

1) *Algorithm Overview:* We adopt the coset viewpoint of stabilizer decoding: any error E can be decomposed as $E = s(E)t(s)l(E)$, where the syndrome s fixes $t(s)$ and ambiguity concentrates in the logical/coset choice. To approach the coset-level maximum-likelihood decision $\operatorname{argmax}_L \{p(L|s)\}$, we

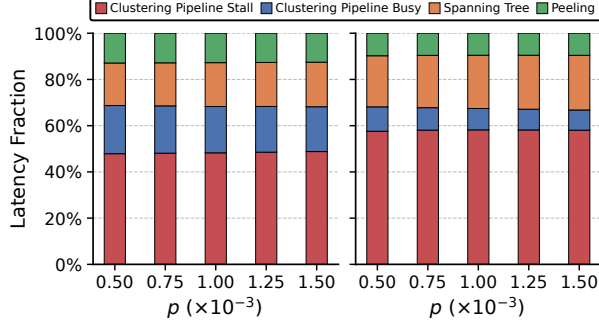


Fig. 3. Latency breakdown for code distances 3 (left) and 11 (right) on a two-stage baseline without optimizations. Clustering Pipeline Stall and Clustering Pipeline Busy are cycles spent inside the *pipelined clustering engine* (stalled vs. productive); Spanning Tree and Peeling are cycles spent in the *post-clustering traversal modules*.

Algorithm 1 Coset Ensemble Decoder

Require: Syndrome parity s ; decoding graph $G = (\mathcal{V}, \mathcal{E})$; candidate number K ; seeds
Ensure: Final correction $\hat{E} \subseteq \mathcal{E}$
Phase I: Clustering
1: $\hat{G} \leftarrow \text{CLUSTERING}(G, s)$
Phase II: Ensemble Forest Exploration
2: $E \leftarrow \emptyset, L \leftarrow \emptyset$
3: **for** $i = 1$ **to** K **do**
4: **for all** $(v, e) \in \hat{G}$ **do**
5: $\phi(v, e) \leftarrow \text{HashToUnit}(\text{seed}, i, v, e)$
6: $(\text{parent}, \sigma) \leftarrow \text{PRIORITYFORESTS}(G, \phi)$ $\triangleright$ Algorithm 2
7: $\{E_i, L_i\} \leftarrow \text{ROE}(\text{parent}, \sigma, s)$ $\triangleright$ Algorithm 3
8: $E \leftarrow E \cup \{E_i\}; \quad L \leftarrow L \cup \{L_i\}$
9: $\hat{E} \leftarrow \text{MAJORVOTE}(E, L)$ $\triangleright$ on smallest- $|E_i|$ subset
10: **return** $\hat{E}$

sample a keyed *priority function* ϕ over vertices and incident edges, construct one deterministic forest per priority sample, and obtain its candidate correction through linear-time ROE. Repeating this process over K independent samples yields coset-consistent candidates whose logical outcomes are aggregated by voting, as shown in Algorithm 1 and Algorithm 2.

2) *Proof of Approximation to Optimality:* As shown in Eq. 6, the optimal ML decoding over logically equivalent cosets is computationally challenging. This is because the size of the Abelian group generated by the stabilizers grows exponentially with the number of stabilizers, and calculating the maximum-likelihood requires summation over this exponentially large group. Our approach approximates this optimal decoding by first partitioning the stabilizer group via clustering and then solving the sub-optimal coset decoding problem over the resulting partitioned cosets. To formalize this approximation, let's first introduce the following definitions and premises:

Definition 1 (Syndrome graph). *We define a syndrome graph as an undirected connected graph $G(\mathcal{V}, \mathcal{E})$ such that $\mathcal{V}$ can be partitioned into $\mathcal{V}_t$ and $\mathcal{V}_{nt}$ such that $\mathcal{V}_t = \{v | s(v) = +1\}$ is called the set of trivial syndromes, $\mathcal{V}_{nt} = \{v | s(v) = -1\}$*

Algorithm 2 PriorityForests

Require: Graph $G = (\mathcal{V}, \mathcal{E})$; Priorities $\phi : \mathcal{V}, \mathcal{E} \rightarrow (0, 1)$
Ensure: Array $\text{parent} : \mathcal{V} \rightarrow \mathcal{V} \cup \{\text{NIL}\}$; discovery order σ
1: $\text{visited}[v] \leftarrow \text{false}, \text{parent}[v] \leftarrow \text{NIL}$ for all $v \in \mathcal{V}$; $\sigma \leftarrow []$; $Q \leftarrow \text{QUEUE}$
2: $\Pi_{\mathcal{V}} \leftarrow \mathcal{V}$ sorted by ascending ϕ
3: **for all** $u \in \Pi_{\mathcal{V}}$ **do**
4: **if not** $\text{visited}[u]$ **then**
5: $\text{ENQUEUE}(Q, u)$; $\text{visited}[u] \leftarrow \text{true}$; $\text{PUSH}(\sigma, u)$
6: **while not** $\text{EMPTY}(Q)$ **do**
7: $x \leftarrow \text{DEQUEUE}(Q)$
8: $\text{Adj} \leftarrow \text{ADJ}(G, x)$ sorted by ascending
9: **for all** $y \in \text{Adj}$ **do**
10: **if not** $\text{visited}[y]$ **then**
11: $\text{parent}[y] \leftarrow x$; $\text{visited}[y] \leftarrow \text{true}$;
 $\text{ENQUEUE}(Q, y)$; $\text{PUSH}(\sigma, y)$
12: **return** (parent, σ)

Algorithm 3 Reverse-Order Elimination (ROE)

Require: Array parent ; discovery order σ ; parity $s \in \{0, 1\}^V$
Ensure: Correction $E_i \subseteq \mathcal{E}$
1: $E_i \leftarrow \emptyset$; $p \leftarrow s$
2: **for** $t = |\sigma|$ **down to** 1 **do**
3: $x \leftarrow \sigma_t$; $r \leftarrow \text{parent}[x]$
4: **if** $r \neq \text{NIL}$ **and** $p[x] = 1$ **then**
5: $E_i \leftarrow E_i \cup \{(x, r)\}$
6: $p[x] \leftarrow p[x] \oplus 1$; $p[r] \leftarrow p[r] \oplus 1$
7: $L_i = \text{DECODELOGICAL}(E_i)$
8: **return** $\{E_i, L_i\}$

the set of non-trivial syndromes, and the size of non-trivial syndromes is even. If the size of non-trivial syndromes is zero, we call the syndrome graph trivial. The input decoding graph of our algorithm is a non-trivial syndrome graph.

Definition 2 (Clustering). *Clustering takes in the input decoding graph $G(\mathcal{V}, \mathcal{E})$ and outputs a partition of sub-graphs $\mathcal{C} = \{G_i(\mathcal{V}_i, \mathcal{E}_i)\}$ such that each G_i is a syndrome graph and precisely one G_i is trivial.*

Premise 1. *After clustering, each non-trivial syndrome graph is sent to ensemble-forest-exploration. The K independent priority samples induce K error ensembles $\{E_i, L_i\}_{i=1}^K$, where $E_i \in \mathcal{P}_n$ and L_i is the corresponding logical error.*

We now present two lemmas to substantiate the claim that our algorithm solves a sub-optimal coset ML problem. The first lemma demonstrates that the errors of candidate error ensembles are degenerate, forming logically equivalent cosets. The second lemma analyzes the algorithm's asymptotic optimality within the partitioned solution space after clustering.

Lemma 1. *For K error ensembles $\{E_i, L_i\}$, the E_i with equal L_i are degenerate errors and belong to the same logical equivalent coset of the stabilizer group S .*

Proof. As shown in Eq. 4, $s(E)$ is a stabilizer term that could deform the error chain, or, equivalently, affect the matching in surface code. $t(s)$ corresponds to the pure error operator, which only depends on the syndrome measurement s , and $l(E)$ is the operator of the E 's logical error. Therefore, for any two

error ensembles $\{E_1, L\}$ and $\{E_2, L\}$ with the same logical error, they both derive from the same syndrome measurement pattern, so their error expansions form the same $t(s)$ and $l(E)$. Because L , S , and $t(s)$ are all Hermitian and unitary,

$$E_1 E_2^\dagger = (S_1 t(s) L) (S_2 t(s) L)^\dagger = S_1 S_2 \quad (7)$$

Therefore, it's proved that for all error ensembles with the same logical error, their errors E_i are degenerate and belong to the same logical-equivalent coset. $\square$

Lemma 2. *The clustering $\mathcal{C}$ reformulates the global optimal coset decoding problem $\arg\max_L \{p(L|s)\}$ by solving a locally optimal Maximum-Likelihood problem*

$$\arg\max_L \left\{ \sum_{b \in \mathcal{B}_c} p(E = S(b)t(s)L) \right\} \quad (8)$$

where for index c running over all non-trivial syndrome graphs in the cluster,

$$\mathcal{B}_c = \{b \in \mathbb{F}_2^m | b_g = 0 \text{ for all } g \text{ with } s_g \notin G_c\} \quad (9)$$

is the set of all m -bit strings whose support is restricted to the indices in the cluster.

Proof. Given a stabilizer code and the syndrome measurement result s , the optimal decoding is to find the logical-equivalent coset with the highest probability by solving Eq. 6. Since the operator of each stabilizer generator S_g can be constructed as $S_g = U Z_g U^\dagger$, and each stabilizer in $\mathcal{S}$ is the product of several stabilizer generators, the stabilizer S can be written as:

$$\begin{aligned} S &= U Z_m^{b_m} U^\dagger U Z_{m-1}^{b_{m-1}} U^\dagger \dots U Z_1^{b_1} U^\dagger \\ &= U \left(Z_m^{b_m} Z_{m-1}^{b_{m-1}} \dots Z_1^{b_1} \right) U^\dagger, \quad b_g \in \{0, 1\} \end{aligned} \quad (10)$$

If $b_g = 1$, the g -th stabilizer generator is multiplied by the pure error term, which would cause a local deformation on the error chain. Therefore, if an m -bit bitstring b is constructed by concatenating b_g as $b = \bigoplus_{g=1}^m b_g \cdot (1 \ll (g-1))$, the error operator will only depend on the value of this bitstring and can be rewritten as $E = S(b)t(s)L$. The activation of g -th bit of b represents the contribution of S_g on deforming the final error chain. In this case, the original coset probability becomes:

$$\sum_{S \in \mathcal{S}} p(E = S t(s) L) \equiv \sum_{b \in \mathbb{F}_2^m} p(E = S(b) t(s) L) \quad (11)$$

After clustering $\mathcal{C}$, the error chain could only be modified locally within each cluster. This implies that only the non-trivial (activated) syndromes within a cluster contribute to its error deformation. Given the one-to-one correspondence between each bit b_g of b , a syndrome bit s_g , and a stabilizer S_g , b_g can vary (0 or 1) or deactivate (set to 0) if its corresponding syndrome s_g is inside or outside any cluster $\mathcal{G}_c$. Each cluster thus defines a local configuration. The space of valid bitstrings is a subset of $\mathbb{F}_2^m$, which is the union of the spaces $\mathcal{B}_c$ as given in (9). In this framework, the clustering $\mathcal{C}$ approximates the original global optimization problem from a locally optimal

version by partitioning the stabilizer space into activated and deactivated regions. $\square$

Based on the preceding lemmas, the error ensembles produced contain degenerate errors that can be grouped into logically equivalent cosets based on their logical errors. Under the priority-sampling distribution, the sample frequency of a logical outcome estimates its probability mass within the partitioned candidate space, which can be represented as $\tilde{p}(L_i|s) = \frac{n_{L_i}}{K}$, where n_{L_i} is the number of ensembles with logical error L_i . The MAJORITYVOTE thus identifies the most frequently sampled coset and approximates the sub-optimal coset ML problem as

$$\arg\max_{L_i} \{\tilde{p}(L_i|s)\} = \arg\max_{L_i} \left(\frac{n_{L_i}}{K} \right) \quad (12)$$

A final correction can then be chosen arbitrarily from this coset due to the degeneracy among candidates. Moreover, as the candidate number $K \rightarrow \infty$, the sampling estimate converges within this partitioned candidate space, though performance remains bounded below the original optimal coset ML due to the clustering constraint. In practice, this vote is restricted to the candidates with the smallest correction size $|E_i|$. This empirical refinement improves accuracy and reduces to Eq. (12) when all candidates share the same correction size.

3) *Relationship to UF and MWPM:* Our coset ensemble decoder typically sits between UF and MWPM. It leverages UF's efficient clustering but critically advances it by introducing a coset-decoding step, which produces multiple error ensembles to identify the most probable logical coset and thus outperforms UF. Directly contrasting with MWPM reveals the impact of the clustering stage. MWPM performs maximum-likelihood (ML) decoding on physical errors, whereas our coset ensemble decoder solves a constrained version of the coset maximum-likelihood problem, and the solution space, a subset of all cosets defined by the clustering, is sub-optimal. Consequently, our decoder may exceed MWPM's accuracy only when the cluster structure aligns with the error structure that MWPM's *Blossom* would capture.

B. Reverse-Order Elimination (ROE)

Given the parent array and discovery order σ from Sec. III-A, ROE scans vertices in the exact reverse of σ and pops vertices in order as shown in Algorithm 3. This eliminates global leaf detection and degree recomputation, delivering a single-pass, linear-time peeling that helps reduce the decoding latency.

The key observation is that Algorithm 2 has already traversed the graph from roots to leaves during forest construction. By recording this order and popping vertices in reverse, ROE reuses that traversal and avoids an additional pass for leaf discovery.

C. Lossless Graph Compression

To reduce the additional exploration efforts introduced by Sec. III-A, we apply structure-preserving reductions with smaller complexity. The example in Fig. 4 illustrates how we

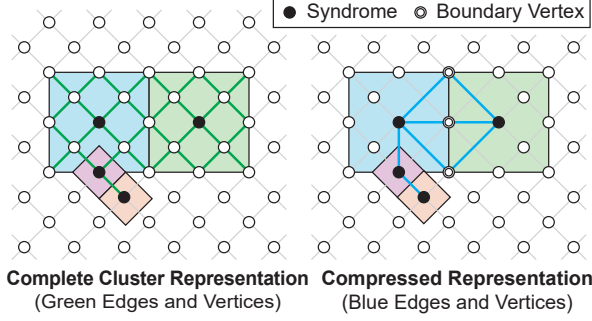


Fig. 4. Graph compression.

obtain the compressed graph structure after clustering. The four colored regions represent the clusters grown from four initial root vertices. On the left, all vertices and green edges inside the colored regions constitute the input graph $G(\mathcal{V}, \mathcal{E})$ of Algorithm 1. Since the complexity of Algorithm 1 is linear in the size of the input graph, graph pruning that preserves its structural information helps reduce the decoding latency.

In this work, we use the compressed graph structure shown on the right. Unlike the complete graph on the left, we retain only the edges between roots and the edges between roots and boundaries during merging. This edge representation, which goes beyond axis-aligned Manhattan connections and allows edges to link vertices in arbitrary directions across the grid, together with the pruning of redundant edges, preserves the core structure of the graph while remaining fully compatible with the main dataflow of Algorithm 1. In Fig. 4, the straightforward representation uses an input graph of size 21, whereas our compressed representation reduces this number to 8.

IV. HARDWARE ARCHITECTURE

A. Hardware Overview

Fig. 5 shows a two-stage architecture aligned with the algorithm: a fully pipelined clustering engine feeds K parallel *Ensemble Forest Exploration (EFE)* instances and a Voting module. Clustering is a streaming growth-and-merge computation that naturally maps to a seven-stage pipeline (S1–S7). Each EFE instance then performs a stateful forest traversal followed by ROE; because this traversal state cannot be time-multiplexed without overwriting in-flight adjacency data, the K instances are replicated and run in parallel. Adjacency-list construction overlaps with clustering, and after clustering terminates each EFE instance traverses under a distinct priority scheme before the Voting module aggregates the predicted logical outcomes.

The main architectural bottleneck lies in the clustering pipeline: as profiled in Fig. 3, pipeline stalls dominate decoding latency. Clustering has low arithmetic intensity and is driven by indirect metadata accesses, so concurrent updates to shared global data can trigger severe bank conflicts. The remainder of this section therefore focuses on the conflict-

aware multi-bank hashed memory system and hierarchical ID mapping scheme that keep this pipeline busy.

B. Fully-Pipelined Architecture for Clustering Stage

These stalls primarily stem from RAW (Read-After-Write) hazards during sequential growth and memory bank conflicts under concurrent updates, motivating our specialized 7-stage pipeline in Fig. 5 with targeted mechanisms to maximize hardware utilization.

The 7-stage pipeline processes one Vertex ID (VID) per cycle to achieve high-throughput decoding. To handle concurrent merge operations, we introduce a **hierarchical ID mapping** that interposes a Root-ID (RID) between VIDs and Cluster-IDs (CIDs). This indirection layer decouples physical vertex storage from logical cluster states. The dataflow operates as follows: (S1–S3) The pipeline dequeues a VID and concurrently fetches its associated RIDs and edge weights to resolve CIDs. (S4) Grow/merge logic is evaluated based on the retrieved metadata. (S5–S7) The pipeline manages active CIDs via a priority-based FIFO and updates boundary-vertex states. As detailed in Sec. IV-D, this RID-based hierarchy is key to collapsing write fan-out and mitigating peak memory bandwidth pressure.

To address the identified bottlenecks, we implement two core optimizations: (1) a **forwarding/bypass network** that feeds S4’s growth decisions back to earlier stages to resolve data dependencies, and (2) a **hash-based memory layout** that minimizes contention during concurrent metadata lookups.

C. Multi-Bank Memory Layout and Hashing Scheme

To handle highly concurrent local memory accesses during cluster growth, we design a customized multi-bank memory with conflict-free hashing. The hashing maps vertex and edge memory accesses to different memory banks, supporting single-cycle access for distances up to 15^1 .

Our memory system must satisfy a critical requirement: for any input lattice coordinate (x, y, z) , we need to simultaneously access (i) the center vertex and its axis-aligned neighbors, and (ii) all incident edges. The core design principle is to ensure that these concurrent accesses land in distinct banks as shown in Fig. 6.

We represent the 3D lattice using integer coordinates (x, y, z) . For edges, we use a consistent convention: each edge is identified by the coordinate of its *positive (forward) endpoint*. Specifically:

$$\pm \mathbf{e}_i : (x, y, z) \leftrightarrow (x, y, z) \pm \mathbf{e}_i, \quad i \in \{x, y, z\} \quad (13)$$

are all represented by (x, y, z) . Our implementation supports lattices with code distance up to 15, with smaller configurations also supported.

To distribute vertex data uniformly across banks, we employ a linear congruential hash function that maps each vertex coordinate to a bank index b_v :

$$b_v(x, y, z) = (\alpha x + \beta y + \gamma z) \bmod M, \quad (14)$$

¹Longer distances incur slightly higher clock cycle overhead.

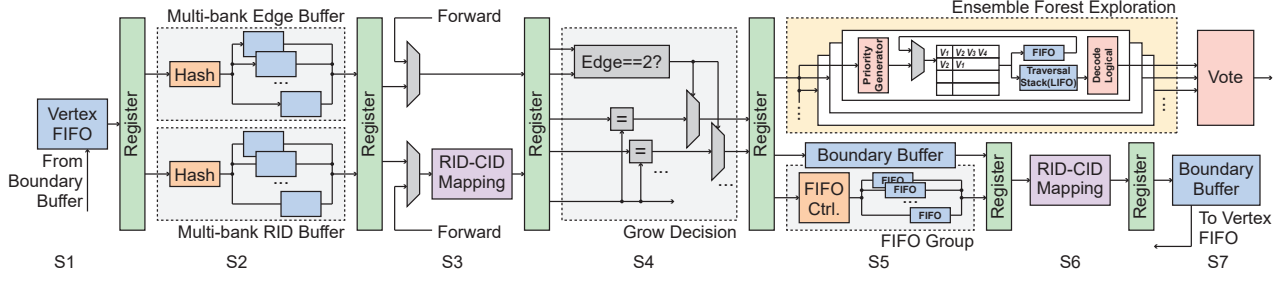


Fig. 5. Two-stage hardware architecture: a fully pipelined clustering engine (outside the light-yellow region) feeds the post-clustering modules (inside the light-yellow region), comprising K parallel Ensemble Forest Exploration (EFE) instances and a Voting module that aggregates the K candidate corrections into the final logical-error estimate.

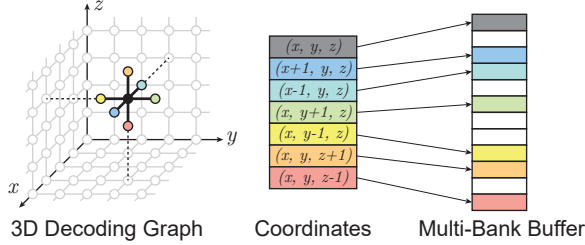


Fig. 6. Multi-bank hashing distributes 7-vertex neighborhood to distinct banks.

where $\alpha = 1, \beta = 3, \gamma = 5$, and $M = 22$. A key property of these coefficients is that the resulting bank indices for the center (x, y, z) and its axis-aligned neighbors are *pairwise distinct by construction*, guaranteeing conflict-free concurrent accesses.

To support this bank distribution, vertices belonging to the same bank are densely packed using a lexicographic traversal order (i, j, k) with i outermost, then j , then k , where $i = 0, \dots, L-1$, $j = 0, \dots, L-1$, $k = 0, \dots, R-1$. The bank-internal address a_v of a vertex is the *rank* of (x, y, z) among all triples that hash to the same bank:

$$a_v(x, y, z) = \sum_{\substack{0 \leq i, j < L \\ 0 \leq k < R}} \left[\left[(i + 3j + 5k) \bmod 22 = b_v(x, y, z) \right] \cdot \left[(i, j, k) \prec_{\text{lex}} (x, y, z) \right] \right] \quad (15)$$

where $[\cdot]$ is the Iverson bracket and $(i, j, k) \prec_{\text{lex}} (x, y, z)$ denotes lexicographic precedence.

D. Hierarchical ID Mapping for Cluster Merging

Another source of memory conflicts happens during cluster merging. It features higher concurrency and weaker spatial locality. Consider the conventional merging process on a 3D grid with $O(N^3)$ points, where each coordinate stores a CID. During cluster growth, only a small, spatially contiguous set of points changes its CID. Our existing multi-bank buffer with hash-based placement handles these localized updates efficiently. In contrast, merges between concurrently growing clusters induce many logically simultaneous CID updates

that may be scattered across the volume. Directly rewriting the “VID $\rightarrow$ CID” store for tens of randomly located VIDs per merge scales poorly in hardware, amplifying both bank conflicts and write bandwidth demands. Fig. 7 (a) presents a straightforward way of changing storage mapping relationships. Each thin line represents a storage mapping relationship. Under the straightforward approach, all storage cells whose CID was originally 3, 6, or 7 must be remapped to 1 during cluster merging, resulting in a total of 15 storage cells that need to be updated.

We introduce an intermediate representation that decouples high-fan-out, poorly localized merge updates from the coordinate address space. VIDs are first mapped to RIDs in the multi-bank, hash-partitioned buffer (optimized for growth). A compact memory then holds an “RID $\rightarrow$ CID” indirection, where CID is the post-merge cluster identifier. Merge operations update only this “RID $\rightarrow$ CID” mapping: all elements formerly addressed by the merged RIDs are logically relabeled by modifying a small number of RID entries rather than rewriting the many coordinates that reference them. Because “VID $\rightarrow$ RID” is already a many-to-one mapping during growth, the merge stage’s write fan-out collapses from “number of touched VIDs” to “number of touched RIDs,” enabling single-cycle remaps in the specialized memory and sharply reducing peak concurrent traffic. In Fig. 7 (b), we only update the memory mapping from RID to CID. In this example, the memory cells storing mapping relationships of RID 3, 6, and 7 are updated. Compared with the straightforward method, the concurrent memory access pressure has been relieved.

V. EXPERIMENTAL METHODOLOGY

A. Experimental Setup

We evaluate the performance of our decoder from a comprehensive perspective, including accuracy, latency, and hardware efficiency. The proposed hardware design is implemented in SystemVerilog HDL on a Xilinx Virtex UltraScale+ VU19P FPGA. The hardware resources and frequency are reported from Vivado 2024.2. The algorithm performance evaluation is conducted through a Python-based hardware simulator, which is cross-validated against our hardware design. It reports logical error rates and cycle counts, and tracks memory-access

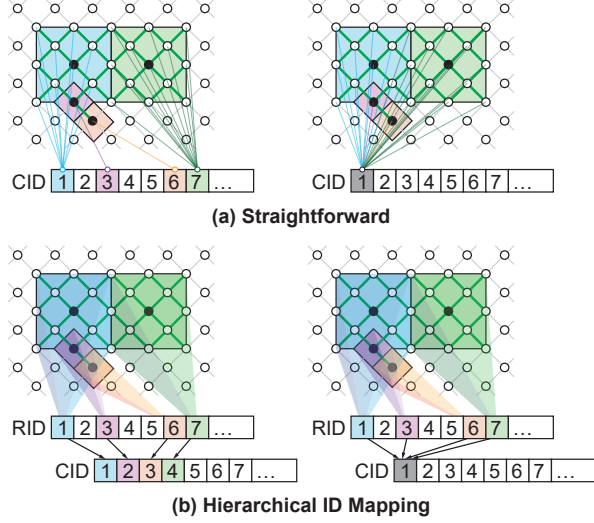


Fig. 7. Comparison of memory-cell update counts between the straightforward method and our method (left: before merging, right: after merging).

conflicts under our multi-bank memory layout and hashing scheme.

Our experiments adopt several widely-used noise models to illustrate the generality of our decoder. (1) Circuit-level depolarizing noise model implemented using the Stim library [22]. For a given code with distance d and a specified number of syndrome extraction rounds, we generate noisy circuits in which depolarizing noise with rate p is applied to data qubits after Clifford operations and between successive rounds of the circuit. Measurement errors are modeled as classical bit flips on the measurement outcomes with the same probability (p), while qubit reset operations are assumed to be ideal. Unless otherwise specified, we use $q = p$ and set the number of repeated syndrome rounds to $T = d$. (2) Biased and unbiased Phenomenological noise model. For biased phenomenological noise, X- and Z-type data faults are injected with probabilities p_X and p_Z , respectively, with bias ratio $\eta = p_Z/p_X$; measurement faults follow the same phenomenological model as above.

All algorithmic accuracy results in this paper are obtained on a surface code with periodic boundary conditions, the same setting used by QUEKUF [23]. For Micro-Blossom [8] and Helios [9], hardware-resource numbers are taken from their original publications on the rotated variant, while decoding latencies are reproduced by running their source code under matched noise conditions. Surface-code variants share the same threshold and differ only in boundary conditions [24]. Reproducing these baselines on a periodic-boundary surface code would increase their decoding latency, since the corresponding syndrome graph is larger; the reported values therefore provide a best-case estimate of these baselines.

B. Decoder Performance Metric

1) *Real-time Compliance*: Modern quantum-classical systems impose tight decoding-latency constraints to prevent backlog, which would otherwise compromise logical fidelity and stall program execution. Prior architecture works for superconducting platforms commonly target sub-microsecond decoding [8], [19], [25]. Following these works, the real-time compliance of hardware decoders is set to the time of one syndrome extraction round.

2) *System Infidelity*: Decoding in fault-tolerant quantum computing (FTQC) can be broadly categorized into two types:

- 1) **Pauli-frame decoding**: the decoding outcome is used solely to correct the measurement result of the corresponding logical qubit through Pauli frame updates. This is typical in memory experiments.
- 2) **Feedback decoding**: the decoding result not only corrects the measurement of a logical qubit but also serves as feedback to conditionally apply logical operations on other qubits, common in implementing non-Clifford operations.

To evaluate Pauli-frame decoding, metrics such as the *logical error rate (LER)* and *reaction time (latency)* [12] are generally sufficient. However, in *Feedback decoding*, the combination of decoding latency and accuracy becomes critical. For instance, suppose a logical operation on logical qubit A is conditioned on the outcome of a Z -basis measurement on logical qubit B . If the decoding of B takes R rounds of syndrome measurements (measured in cycles), then the physical qubits of A must remain idle during this time. As a result, R additional rounds of memory decoding must be applied to A before the conditional operation can proceed. However, this waiting increases the total logical error rate since more errors would be accumulated on physical qubits as discussed in [26]. Therefore, a fairer and more appropriate metric is required to evaluate the impact of decoding latency on decoding accuracy in feedback-based logical operations. We defined this metric to quantify how the decoding latency of logical patch B affects the decoding fidelity of logical patch A , specifically when A 's operation is conditioned on B 's mid-circuit measurement result.

In Ref. [26], the decoder error rate $E(n)$ after n rounds of syndrome measurements assuming a per-round logical error rate ϵ is given by an empirical formula $E(n) = \frac{1}{2}(1 - (1 - 2\epsilon)^n)$. However, ϵ is not directly measurable in FTQC, where the fundamental unit is a full QEC cycle of d syndrome rounds. We therefore reparametrize $E(n)$ in terms of the decoder's LER over d rounds, $E(d)$. Using $(1 - 2\epsilon)^d = 1 - 2E(d)$ and $(1 - 2\epsilon)^n = ((1 - 2\epsilon)^d)^m$ with $m = n/d$ the number of decoding cycles, we obtain the *effective decoder error rate*

$$\hat{E}(m) = \frac{1}{2}(1 - (1 - 2E(d))^m) \quad (16)$$

and the corresponding *effective decoder fidelity*

$$\hat{F}(m) = 1 - 2\hat{E}(m) = (1 - 2E(d))^m. \quad (17)$$

In a feedback-decoding scenario, if the decoding latency for qubit B is R (in units of syndrome cycles) and qubit A has

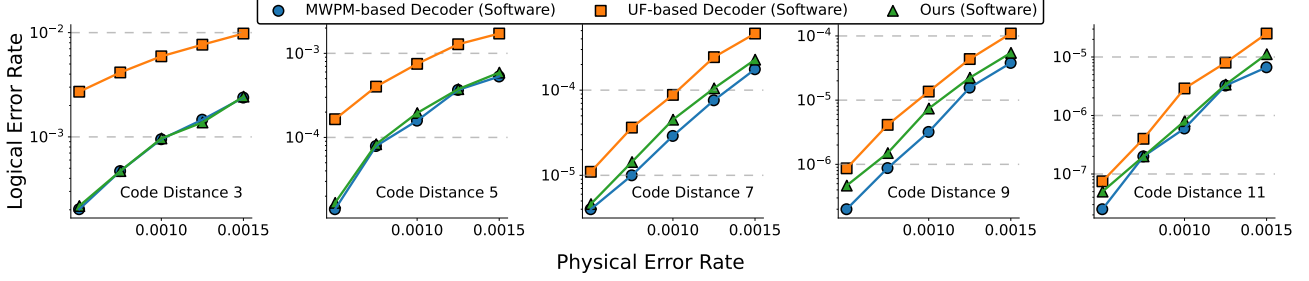


Fig. 8. Logical error rate comparison among MWPM-based decoders, UF-based decoders, and our decoder.

been idle for m decoding cycles, the effective fidelity of A under B 's latency becomes

$$\hat{F}\left(m + \frac{R}{d}\right) = (1 - 2E(d))^{R/d} \cdot \hat{F}(m), \quad (18)$$

where R is computed from B 's decoding latency and $E(d)$ is A 's decoding LER. The impact of B 's latency is thus captured by the factor $(1 - 2E(d))^{R/d}$. If the decoding latency is shorter than one syndrome cycle, no backlog occurs and the LER is unaffected, so R is floored at 1. Inverting for convention (lower is better), the resulting *Infidelity factor* is

$$\hat{C}(R) = 1 - (1 - 2E(d))^{\frac{\max(1, R)}{d}} \in [0, 1), \quad (19)$$

with $R = L/l$ where L is the decoding latency and l the duration of one syndrome extraction round; the mask $\max(1, R)$ implies that if B 's latency is less than one extraction round, its impact on A 's fidelity is negligible and $\hat{C}(R)$ is dominated by $E(d)$. This threshold is sufficient for FTQC because as long as decoding completes before the next syndrome is extracted, latency does not degrade the LER. A lower $\hat{C}(R)$ indicates higher fidelity under latency constraints. While sensitive to both idle time and idle error rates, the impacts of physical idle errors, along with optimizations such as Dynamic Decoupling and Pauli Twirling, are fully encapsulated by $E(d)$.

VI. END-TO-END EVALUATION

We first evaluate algorithmic accuracy, latency, and system-level impact, and then analyze the hardware cost and scalability of the proposed design.

A. Decoding Accuracy Evaluation

The decoding accuracy of the proposed method is evaluated under the circuit-level noise model described in Sec. V-A. To isolate the algorithmic gain introduced by the ensemble-forest method in Sec. III-A, we first compare against two widely used surface-code decoders, MWPM and UF, at the algorithm level as shown in Fig. 8. For the accuracy estimate of MWPM-based decoders, we use the PyMatching implementation [27]. For the UF-based decoders Helios and QUEKUF, we evaluate accuracy using our own baseline UF software implementation to avoid conflating decoder quality with minor differences in boundary-condition handling across implementations.

In these experiments, the candidate number K in our design is fixed to 24. When the code distance is small, the accuracy

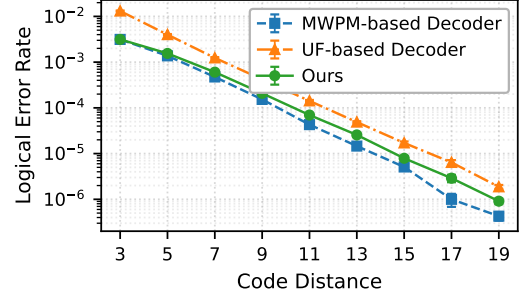


Fig. 9. Logical error rate of MWPM, UF, and our decoder ($K=24$) vs. code distance at $p=0.002$ circuit-level noise.

of our coset ensemble decoding is close to that of MWPM. For larger code distances, the increased graph size causes this fixed K to limit further accuracy improvements.

To examine how the accuracy advantage persists at larger code distances, Fig. 9 reports the logical error rate of MWPM, UF, and our decoder at a fixed physical error rate $p=0.002$ circuit-level noise for $d \in \{3, 5, \dots, 19\}$. Our decoder tracks MWPM closely across this range, substantially outperforming UF: the LER ratio to MWPM grows from $1.0\times$ at $d=3$ to $\sim 2.1\times$ at $d=19$ at fixed $K=24$; the residual gap can be further reduced by increasing K .

B. Decoding Latency Evaluation

Fig. 10 compares the average decoding latency per d -round task of our decoder against Micro-Blossom and Helios for $d = 3$ to $d = 11$. The MWPM-based Micro-Blossom grows steeply with d , reflecting the high complexity of minimum-weight matching, while our decoder and Helios both remain sub-microsecond across all evaluated distances.

Helios's distributed per-vertex PE array yields a latency that scales sublinearly with d through per-iteration coordination and convergecast, whereas our pipelined design scales with the active-vertex count while keeping the hardware footprint compact (the 24 ensemble candidates are processed in parallel and do not affect the critical path). At small d , where active vertices are few, our pipeline retires the task well below Helios's per-iteration floor, producing the $3\text{--}5\times$ advantage at $d=3$. As d grows, the two curves converge near $d=7$ –

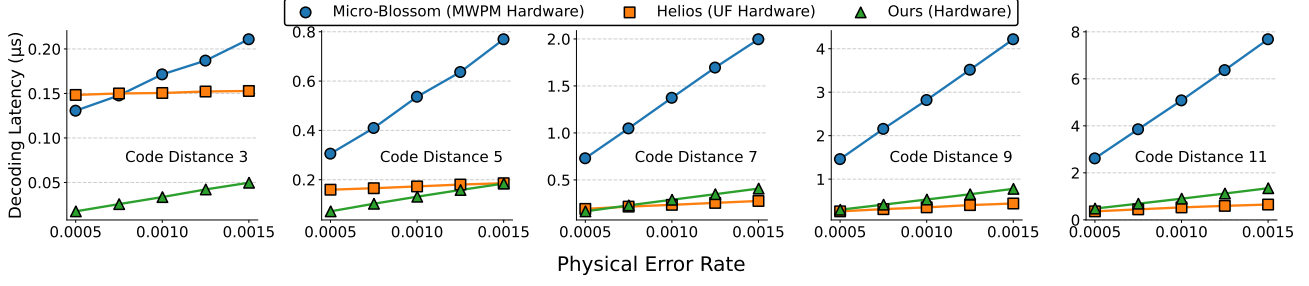


Fig. 10. Decoding latency per decoding task (d syndrome rounds), compared with state-of-the-art decoders.

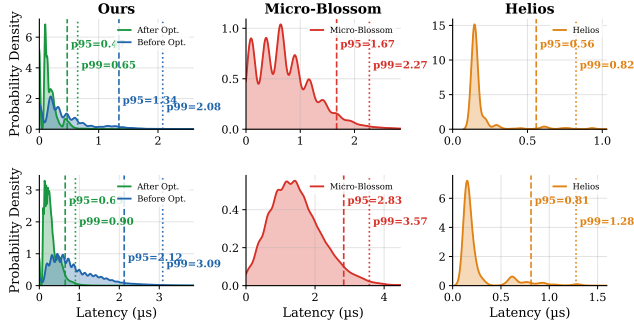


Fig. 11. Decoding latency distribution per d -round task at $p=0.0005$, for code distance 7 (top) and 9 (bottom).

9, with our design still ahead in lower- p settings at $d=7$; beyond this range, Helios's sublinear scaling becomes more favorable on pure latency. Our design targets a resource-efficient latency/area point, using roughly $6\times$ fewer LUTs and $3\times$ fewer FFs than Helios at $d=15$ (Table I). Throughput follows the same pattern, ranging from 1.88 M decodes/sec at $d=9$ to 29.8 M at $d=3$ (both at $p=0.001$), which is $4\text{--}5\times$ Micro-Blossom and comparable to Helios.

Fig. 11 presents the latency probability density per full d -round decoding task. Our optimizations reduce not only average latency but also tail latency: at $d=9$, p_{95} drops from $2.12\mu\text{s}$ to $0.65\mu\text{s}$ (-69%) and p_{99} from $3.09\mu\text{s}$ to $0.90\mu\text{s}$ (-71%), with comparable improvements at $d=7$. The narrowed density curves illustrate that the optimizations reduce variance rather than merely shifting the mean. Compared with the Micro-Blossom, our optimized p_{99} is $3.5\text{--}4\times$ lower (0.65 vs. $2.27\mu\text{s}$ at code distance 7; 0.90 vs. $3.57\mu\text{s}$ at code distance 9) and slightly below Helios (0.65 vs. $0.82\mu\text{s}$; 0.90 vs. $1.28\mu\text{s}$). These results show that the proposed decoder achieves competitive or superior tail-latency behavior relative to state-of-the-art MWPM designs.

C. System Infidelity Comparison

Fig. 12 compares our decoder with three baseline designs in terms of system infidelity, a more appropriate metric introduced in Sec. V-B. For small code distances $d = 3$, our decoder achieves essentially the same system infidelity as Micro-Blossom, and significantly outperforms both UF-

based decoders. Although MWPM yields intrinsically lower LER than UF variants, Micro-Blossom's decoding latency exceeds the threshold at $d \geq 5$ (Fig. 10), so $\hat{C}(R) > 0$ penalizes its system infidelity; it eventually becomes worse than that of Helios. In contrast, our decoder maintains low infidelity thanks to its better scalability, achieving higher accuracy than the UF-based decoders while maintaining low latency. At $d = 11$, our decoder reduces the system infidelity by up to 74.3% compared to Micro-Blossom and by 51.7% compared to Helios.

VII. HARDWARE PERFORMANCE ANALYSES

A. Matched/Normalized Hardware Resource Comparison

Table I summarizes the hardware cost of our design and three representative baseline decoders: Micro-Blossom [8], Helios [9], and QUEKUF [23]. To make a fair comparison, we further normalize prior results to a common code distance using the reported resources together with the scaling complexity described in the original papers. Specifically, we estimate the resource cost at the target distance by fitting/interpolating from the reported results under the stated scaling trend. For latency comparisons, the clock frequency of each baseline is taken as published from its single design point, and is applied uniformly across all evaluated code distances; this matches our own setup, in which a single RTL design is used across all distances. In terms of logic utilization, our architecture requires only 108k LUTs, which is about $8.0\times$ fewer than Micro-Blossom, and roughly $4.3\times$ fewer than QUEKUF. A similar trend holds for flip-flops (FFs): our design uses 43k FFs, i.e., $2.9\times$ fewer than Helios and $14.7\times$ fewer than QUEKUF. Regarding on-chip memory, our design uses about half the BRAM of QUEKUF while supporting nearly twice the maximum code distance, which demonstrates a better efficiency. In terms of achievable frequency, our decoder runs at 163 MHz, which is $3.7\times$ higher than Micro-Blossom and $2.1\times$ higher than Helios.

We also quantify the dynamic-power overhead of ensemble parallelism using the Vivado power report. Each EFE branch contributes about 50 mW of dynamic power, so the $K=24$ parallel branches together account for approximately 1.2 W of dynamic power. Increasing K therefore scales only the branch term linearly while leaving the shared clustering engine and voting part unchanged.

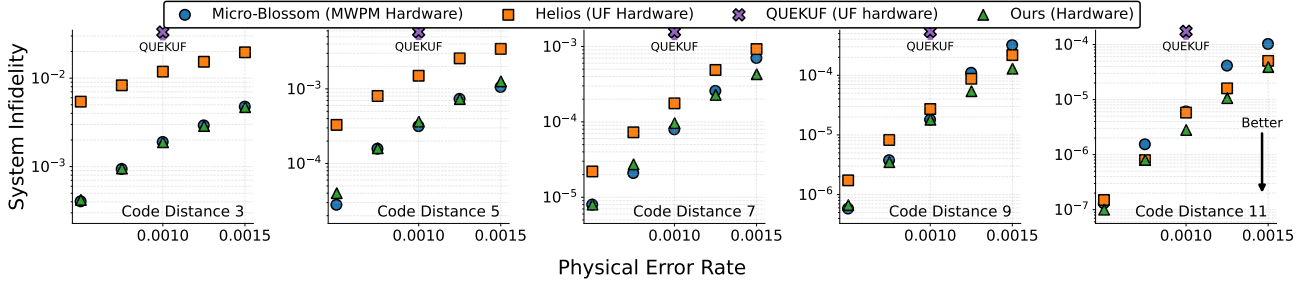


Fig. 12. System infidelity comparison with SOTA decoders.

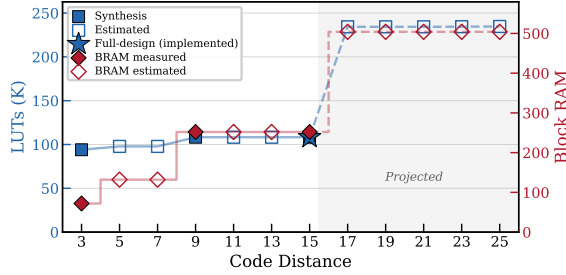


Fig. 13. FPGA resource usage vs. code distance d . Filled markers denote full Vivado synthesis ($d=3, 9, 15$); open markers are estimates. The shaded region indicates extrapolation beyond measured data.

TABLE I
HARDWARE RESOURCE COMPARISON. FOR FAIRER COMPARISON ACROSS DIFFERENT CODE DISTANCES, HELIOS AND QUEKUF ARE SHOWN WITH BOTH THE ORIGINALLY REPORTED VALUES AND NORMALIZED ESTIMATES AT CODE DISTANCE 15.

	Micro-B.	Helios		QUEKUF		Ours
		Orig.	Norm.	Orig.	Norm.	
LUT (k)	867	889	614	309	463	108
FF (k)	NA	177	126	453	634	43
BRAM tiles	3	NA	NA	548	828	252
Freq (MHz)	43	75	NA	238	NA	163
Code distance	15	17	15	8	15	15

B. Hardware Scalability with Code Distance

Existing MWPM- and UF-based hardware decoders typically exhibit rapidly growing resource consumption and degraded clock frequency as the code distance increases. In contrast, our architecture is explicitly designed to scale more efficiently with distance, providing a more hardware-efficient solution toward larger-scale surface-code decoding.

Fig. 13 estimates FPGA resource costs as d scales from 3 to 25. Across this sweep we scale only the components addressed by lattice coordinates—primarily the multi-bank vertex/edge buffer of the clustering engine and the per-EFE adjacency storage—while holding the rest of the design at its measured $d=15$ sizing. The scaled buffers grow analytically as $O(2^{\lceil \log_2 d \rceil})$ due to power-of-two address quantization.

C. Latency Breakdown and Residual Stalls

Fig. 14 breaks down the decoding latency under the same setting of Fig. 10 before and after our optimizations. The clustering stage is pipelined so that its execution overlaps with the subsequent forest-construction and peeling stages; when the pipeline cannot be fed in time, the resulting stalls appear as idle cycles. In the baseline design, stalls in the clustering pipeline dominate execution time, accounting for 48%–58% of total latency across tested configurations. After applying the proposed optimizations, the clustering pipeline stalls fraction drops to 1–7%, effectively eliminating pipeline bubbles. This translates to an overall speedup of $2.2\text{--}3.6\times$ over the baseline, with larger gains at higher code distances (e.g., $3.0\text{--}3.4\times$ at $d=9$ and $3.2\text{--}3.6\times$ at $d=11$). The residual stalls in the clustering pipeline originate from a parity-update RAW hazard in the clustering tail: once only a few active clusters remain in flight, a newly dispatched vertex cannot resolve its merge decision until the preceding growth step commits its parity update, briefly draining the pipeline between dispatches. Eliminating this 1–7% residual would require either speculative parity evaluation or out-of-order dispatch, both of which incur non-negligible control-logic and memory-bandwidth overhead without affecting the logical error rate. We therefore retain the simpler in-order pipeline, which offers a favorable design point given the small remaining headroom.

VIII. SENSITIVITY AND ROBUSTNESS ANALYSIS

A. Tunability Analysis

We fit an empirical power-law model: $\text{LER}(K) = \text{LER}_\infty + A \cdot K^{-\alpha}$, with LER_∞ the error floor, A the improvement headroom, and α the diminishing-returns exponent. Fig. 15(a) shows the fit holds for each (d, p) , with α decreasing from 1.98 ($d=3$) to 0.27 ($d=9$). Defining K^* as the smallest K capturing 70% of the LER improvement yields $K^* = 2^{\lfloor (d+1)/2 \rfloor}$. This remains accurate at $p=0.0015$ for $d \geq 7$, but underestimates K^* for $d \in \{3, 5\}$, since larger p requires more candidates per fractional gain. Fig. 15(b) plots the accuracy–resource Pareto $\text{LUT}_{\text{total}} = \text{LUT}_{\text{fixed}} + K \cdot \text{LUT}_{\text{branch}}$; red stars mark K^* near each knee.

B. Compatibility and Comparison with BP+OSD

To position our decoder on the accuracy spectrum between UF and near-optimal decoders, we include BP+OSD as an

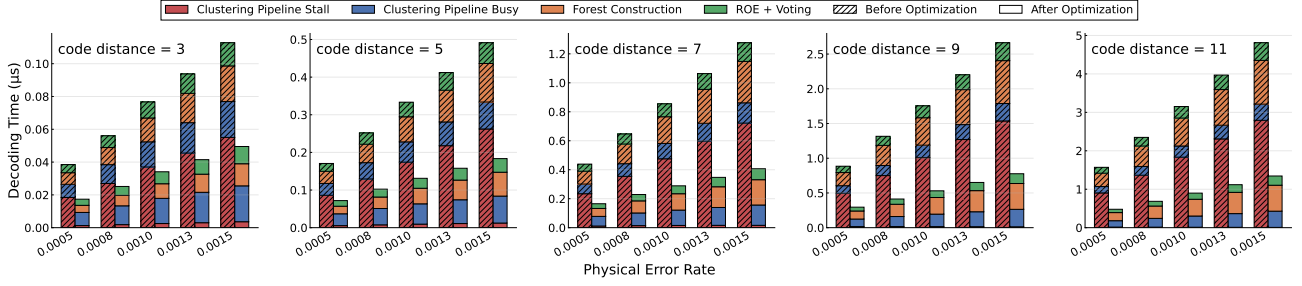


Fig. 14. Hardware latency breakdown before and after optimization.

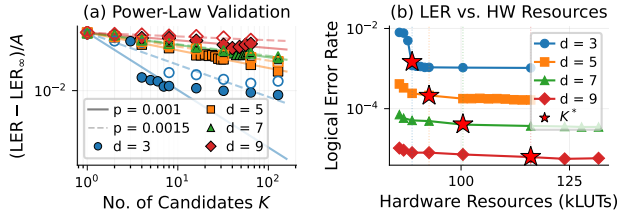


Fig. 15. Tunability analysis of our proposal. (a) Power-law model validation at two noise rates ($p=0.001, 0.0015$). (b) LER vs. hardware area.

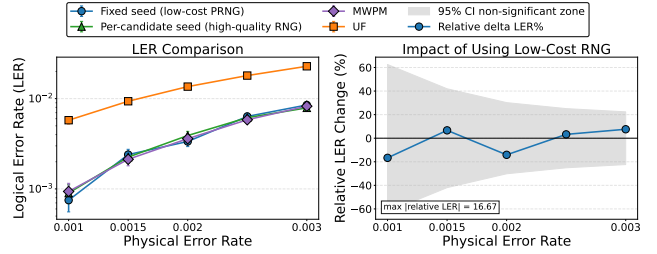


Fig. 18. Decoding performance of different random policies.

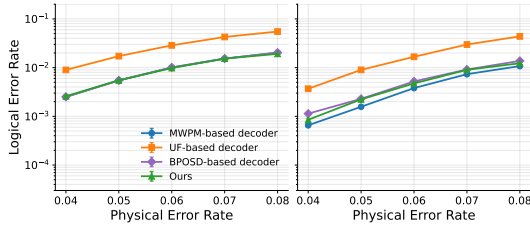


Fig. 16. Logical error rate comparison on the repetition code under a phenomenological noise model.

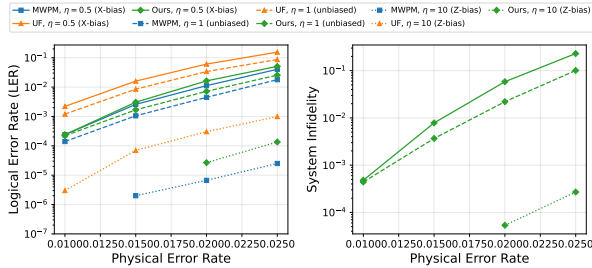


Fig. 17. Logical error rate (left) and the corresponding system infidelity (right) under three biased phenomenological noise settings ($d=7$, X-channel).

accuracy reference alongside MWPM and UF, using product-sum BP with OSD-CS of order 15 on the same Tanner graph for a fair comparison. The evaluation uses the repetition code under a phenomenological noise model, which additionally verifies the generality of our decoder across code families and noise models. We sweep code distances $d \in \{5, 7\}$ and

physical error rates $p \in [0.04, 0.08]$. Fig. 16 reports the LER of MWPM, UF, BP+OSD, and our decoder, with the left and right panels corresponding to $d=5$ and $d=7$, respectively. Our decoder achieves LER within $1.0\text{--}1.4\times$ of MWPM, on par with BP+OSD ($1.0\text{--}1.7\times$), while UF trails by $2.7\text{--}5.7\times$. On these benchmarks, our decoder tracks MWPM and BP+OSD closely, decisively separating it from UF.

We also evaluate our decoder under biased phenomenological noise with three common bias ratios $\eta = p_Z/p_X$: $\eta=0.5$ (X-biased), $\eta=1$ (depolarizing), and $\eta=10$. As shown in Fig. 17, our decoder closes $\sim 94\%$ of the UF-to-MWPM gap under X-biased noise, where vanilla UF incurs $6.2\times$ higher LER than MWPM. The corresponding system-infidelity curves are shown in the right panel of Fig. 17.

C. Robustness to Low-Cost Randomness

A practical concern for FPGA deployment is whether decoder gains rely on high-quality, fully independent random streams, which are often expensive to implement in hardware. To evaluate this risk in a conservative setting, we run all experiments with a *fixed-seed* random policy during decoding. In our implementation, each decoding shot uses a single stateful PRNG stream initialized from a fixed base seed.

We evaluate our decoder under different random policies. Fig. 18 shows the LER comparison and quantifies the relative LER difference between different PRNGs. The shaded region indicates a 95% non-significant zone using a binomial approximation. Our work remains competitive against both MWPM and UF when using cheap but low-quality PRNGs. The relative differences are small and mostly lie within the

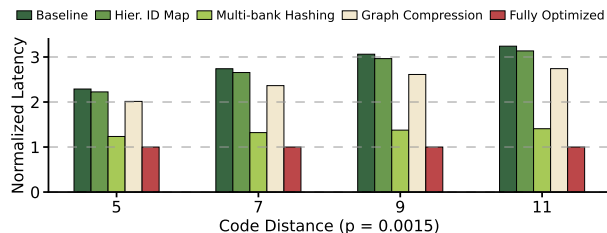


Fig. 19. Decoding latency with and without the proposed optimizations.

95% non-significant band, indicating no meaningful instability from the low-cost fixed-seed setting.

D. Optimization Ablation

We perform an ablation study to quantify the impact of different optimizations. We use a hardware-oriented software simulator that mirrors the dataflow of our final microarchitecture and allows individual hardware features to be selectively enabled or disabled.

We take the coset ensemble decoder architecture without any additional optimization as our baseline. Through ablation experiments, we aim to demonstrate that these optimizations can work synergistically to achieve an overall reduction in decoding latency. The trend in Fig. 19 shows that the benefits of our optimizations increase with larger code distance d . At $d = 11$ and $p = 0.0015$, Hierarchical ID Mapping delivers $1.03\times$, Multi-bank Hashing delivers $2.30\times$, and Graph Compression delivers $1.18\times$ speedup over the baseline; enabling all optimizations achieves a $3.24\times$ overall speedup.

IX. RELATED WORK

A. QEC Algorithm

While there exist families of quantum error correction codes and decoding algorithms [28]–[33], our work focuses on the surface code [34], [35]. Two mainstream decoding methods are Minimum-Weight Perfect Matching (MWPM) [13] and Union-Find (UF) [7]. MWPM solves the physical ML error problem while UF is a faster, sub-optimal version. Our decoder, by accounting for degeneracy and logical cosets, achieves higher accuracy than UF-based decoders in the LER comparisons (Fig. 8 and Fig. 9) while remaining in a similar low-latency regime (Fig. 10); compared with MWPM-based Micro-Blossom, it provides comparable accuracy at significantly lower latency. In contrast, the Tensor-Network (TN) decoder [35], which directly solves the logical coset ML problem, suffers from high contraction complexity. Although our decoder solves a sub-optimal coset ML problem, it maintains very low latency and high scalability for real-time implementation.

B. QEC Hardware and Compilation

There have been various hardware implementations of MWPM and UF decoders and their variants [8], [9], [17], [20], [23], [36]–[39]. In particular, [20], [23] optimize the Union-Find clustering phase by mapping many vertices or

clusters to distributed processing elements and exploiting extensive spatial parallelism, achieving low latency at the cost of considerable hardware resources. By contrast, our design uses a single deeply pipelined clustering data path and explicitly reduces pipeline stalls via forwarding and conflict-aware memory organization, attaining low latency while remaining significantly more resource-efficient. In parallel, recent efforts have sought to automate and accelerate the generation of detector error models for QEC protocols involving logical operations [40], [41].

X. CONCLUSION

This work presents a novel algorithm–hardware co-design for quantum error correction (QEC) decoding. At the algorithmic level, we propose coset ensemble decoding, together with reverse-order elimination and lossless graph compression, to approximate coset-level maximum-likelihood decoding at practical cost. At the hardware level, we introduce a customized architecture with multi-bank memory hashing and hierarchical ID mapping, achieving high hardware efficiency and scalability. Overall, our co-design achieves a better accuracy–latency trade-off than prior state-of-the-art decoders, while maintaining high resource efficiency for scalable QEC decoding. Future work includes assessing the practicality of this approach on available quantum devices, evaluating its performance across broader code families and noise settings, and automating the proposed optimizations across a broader range of FPGA implementations.

ACKNOWLEDGMENT

The support of the UK EPSRC (Grant EP/W03221X/1, EP/V028251/1, EP/S030069/1, EP/X036006/1), UKRI (Grant 256), Altera and AMD is gratefully acknowledged.

REFERENCES

- [1] M. Kjaergaard, M. E. Schwartz, J. Braumüller, P. Krantz, J. I.-J. Wang, S. Gustavsson, and W. D. Oliver, “Superconducting qubits: Current state of play,” *Annual Review of Condensed Matter Physics*, vol. 11, pp. 369–395, 2020.
- [2] B. M. Terhal, “Quantum error correction for quantum memories,” *Reviews of Modern Physics*, vol. 87, no. 2, p. 307, 2015.
- [3] D. Aharonov and M. Ben-Or, “Fault-tolerant quantum computation with constant error rate,” *arXiv preprint quant-ph/9611025*, 1997.
- [4] E. Knill, R. Laflamme, and W. H. Zurek, “Resilient quantum computation: error models and thresholds,” *Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, vol. 454, no. 1969, pp. 365–384, 1998.
- [5] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, “Surface codes: Towards practical large-scale quantum computation,” *Physical Review A*, vol. 86, no. 3, p. 032324, 2012.
- [6] E. Dennis, A. Kitaev, A. Landahl, and J. Preskill, “Topological quantum memory,” *Journal of Mathematical Physics*, vol. 43, no. 9, pp. 4452–4505, 2002.
- [7] N. Delfosse and N. H. Nickerson, “Almost-linear time decoding algorithm for topological codes,” *Quantum*, vol. 5, p. 595, 2021.
- [8] Y. Wu, N. Liyanage, and L. Zhong, “Micro blossom: Accelerated minimum-weight perfect matching decoding for quantum error correction,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 639–654.
- [9] N. Liyanage, Y. Wu, A. Deters, and L. Zhong, “Scalable quantum error correction for surface codes using FPGA,” in *2023 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 1. IEEE, 2023, pp. 916–927.
- [10] J. Roffe, “Quantum error correction: an introductory guide,” *Contemporary Physics*, vol. 60, no. 3, pp. 226–245, Jul. 2019. [Online]. Available: <http://dx.doi.org/10.1080/00107514.2019.1667078>
- [11] D. Poulin and Y. Chung, “On the iterative decoding of sparse quantum codes,” 2008. [Online]. Available: <https://arxiv.org/abs/0801.1241>
- [12] R. Acharya, D. A. Abanin, L. Aghababaie-Beni, I. Aleiner, T. I. Andersen, M. Ansmann, F. Arute, K. Arya, A. Asfaw, N. Astrakhantsev, J. Atalaya, R. Babbush, D. Bacon, B. Ballard, J. C. Bardin, J. Bausch, A. Bengtsson, A. Bيلمes, S. Blackwell, S. Boixo, G. Bortoli, A. Bourassa, J. Bovaird, L. Brill, M. Broughton, D. A. Browne, B. Buchea, B. B. Buckley, D. A. Buell, T. Burger, B. Burkett, N. Bushnell, A. Cabrera, J. Campero, H.-S. Chang, Y. Chen, Z. Chen, B. Chiaro, D. Chik, C. Chou, J. Claes, A. Y. Cleland, J. Cogan, R. Collins, P. Conner, W. Courtney, A. L. Crook, B. Curtin, S. Das, A. Davies, L. De Lorenzo, D. M. Debroy, S. Demura, M. Devoret, A. Di Paolo, P. Donohoe, I. Drozdov, A. Dunsworth, C. Earle, T. Edlich, A. Eickbusch, A. M. Elbag, M. Elzouka, C. Erickson, L. Faoro, E. Farhi, V. S. Ferreira, L. F. Burgos, E. Forati, A. G. Fowler, B. Foxen, S. Ganjam, G. Garcia, R. Gasca, É. Genois, W. Jiang, C. Gidney, D. Gilboa, R. Gosula, A. G. Dau, D. Graumann, A. Greene, J. A. Gross, S. Habegger, J. Hall, M. C. Hamilton, M. Hansen, M. P. Harrigan, S. D. Harrington, F. J. H. Heras, S. Heslin, P. Heu, O. Higgott, G. Hill, J. Hilton, G. Holland, S. Hong, H.-Y. Huang, A. Huff, W. J. Huggins, L. B. Ioffe, S. V. Isakov, J. Iveland, E. Jeffrey, Z. Jiang, C. Jones, S. Jordan, C. Joshi, P. Juhas, D. Kafri, H. Kang, A. H. Karamlou, K. Kechedzhi, J. Kelly, T. Khair, T. Khattar, M. Khezri, S. Kim, P. V. Klimov, A. R. Klots, M. Kobrin, P. Kohli, A. N. Korotkov, F. Kostritsa, R. Kothari, B. Kozlovskii, J. M. Kreikebaum, V. D. Kurilovich, N. Lacroix, D. Landhuis, T. Lange-Dei, B. W. Langley, P. Laptev, K.-M. Lau, L. Le Guevel, J. Ledford, J. Lee, K. Lee, Y. D. Lensky, S. Leon, B. J. Lester, W. Y. Li, Y. Li, A. T. Lill, W. Liu, W. P. Livingston, A. Locharla, E. Lucero, D. Lundahl, A. Lunt, S. Madhuk, F. D. Malone, A. Maloney, S. Mandrà, J. Manyika, L. S. Martin, O. Martin, S. Martin, C. Maxfield, J. R. McClean, M. McEwen, S. Meeks, A. Megrant, X. Mi, K. C. Miao, A. Mieszala, R. Molavi, S. Molina, S. Montazeri, A. Morvan, R. Movassagh, W. Mruczkiewicz, O. Naaman, M. Neeley, C. Neill, A. Nersisyan, H. Neven, M. Newman, J. H. Ng, A. Nguyen, M. Nguyen, C.-H. Ni, M. Y. Niu, T. E. O’Brien, W. D. Oliver, A. Opremcak, K. Ottosson, A. Petukhov, A. Pizzuto, J. Platt, R. Potter, O. Pritchard, L. P. Pryadko, C. Quintana, G. Ramachandran, M. J. Reagor, J. Redding, D. M. Rhodes, G. Roberts, E. Rosenberg, E. Rosenfeld, P. Roushan, N. C. Rubin, N. Saei, D. Sank, K. Sankaragomathi, K. J. Satzinger, H. F. Schurkus, C. Schuster, A. W. Senior, M. J. Shearn, A. Shorter, N. Shutty, V. Shvarts, S. Singh, V. Sivak, J. Skrzynny, S. Small, V. Smelyanskiy, W. C. Smith, R. D. Somma, S. Springer, G. Sterling, D. Strain, J. Suchard, A. Szasz, A. Sztein, D. Thor, A. Torres, M. M. Torunbalci, A. Vaishnav, J. Vargas, S. Vdovichev, G. Vidal, B. Villalonga, C. V. Heidweiller, S. Waltman, S. X. Wang, B. Ware, K. Weber, T. Weidel, T. White, K. Wong, B. W. K. Woo, C. Xing, Z. J. Yao, P. Yeh, B. Ying, J. Yoo, N. Yosri, G. Young, A. Zalcman, Y. Zhang, N. Zhu, and N. Zobrist, “Quantum error correction below the surface code threshold,” *Nature*, vol. 638, no. 8052, pp. 920–926, Dec. 2024. [Online]. Available: <http://dx.doi.org/10.1038/s41586-024-08449-y>
- [13] O. Higgott and C. Gidney, “Sparse blossom: correcting a million errors per core second with minimum-weight matching,” *Quantum*, vol. 9, p. 1600, Jan. 2025. [Online]. Available: <http://dx.doi.org/10.22331/q-2025-01-20-1600>
- [14] D. Gottesman, “Stabilizer codes and quantum error correction,” 1997. [Online]. Available: <https://arxiv.org/abs/quant-ph/9705052>
- [15] P. Fuentes, J. Etzezarreta Martinez, P. M. Crespo, and J. Garcia-Frías, “Degeneracy and its impact on the decoding of sparse quantum codes,” *IEEE Access*, vol. 9, pp. 89093–89119, 2021.
- [16] T. M. Stace and S. D. Barrett, “Error correction and degeneracy in surface codes suffering loss,” *Physical Review A*, vol. 81, no. 2, Feb. 2010. [Online]. Available: <http://dx.doi.org/10.1103/PhysRevA.81.022317>
- [17] Y. Wu and L. Zhong, “Fusion blossom: Fast mwpm decoders for qec,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.08307>
- [18] V. Kolmogorov, “Blossom v: a new implementation of a minimum cost perfect matching algorithm,” *Mathematical Programming Computation*, vol. 1, pp. 43–67, 2009. [Online]. Available: <https://api.semanticscholar.org/CorpusID:17864814>
- [19] P. Das, C. A. Pattison, S. Manne, D. M. Carmean, K. M. Svore, M. Qureshi, and N. Delfosse, “Afs: Accurate, fast, and scalable error-decoding for fault-tolerant quantum computers,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2022, pp. 259–273.
- [20] N. Liyanage, Y. Wu, S. Tagare, and L. Zhong, “FPGA-Based Distributed Union-Find Decoder for Surface Codes,” *IEEE Transactions on Quantum Engineering*, vol. 5, pp. 1–18, 2024. [Online]. Available: <http://dx.doi.org/10.1109/TQE.2024.3467271>
- [21] M.-H. Hsieh and F. Le Gall, “Np-hardness of decoding quantum error-correction codes,” *Physical Review A*, vol. 83, no. 5, May 2011. [Online]. Available: <http://dx.doi.org/10.1103/PhysRevA.83.052331>
- [22] C. Gidney, “Stim: a fast stabilizer circuit simulator,” *Quantum*, vol. 5, p. 497, 2021.
- [23] F. Valentino, B. Branchini, D. Conficconi, D. Sciuto, and M. D. Santambrogio, “QUEKUF: an FPGA Union Find Decoder for Quantum Error Correction on the Toric Code,” *ACM Transactions on Reconfigurable Technology and Systems*, 2025.
- [24] D. S. Wang, A. G. Fowler, A. M. Stephens, and L. C. Hollenberg, “Threshold error rates for the toric and surface codes,” *arXiv preprint arXiv:0905.0531*, 2009.
- [25] A. Holmes, M. R. Jokar, G. Pasandi, Y. Ding, M. Pedram, and F. T. Chong, “Nisq+: Boosting quantum computing power by approximating quantum error correction,” in *2020 ACM/IEEE 47th annual international symposium on computer architecture (ISCA)*. IEEE, 2020, pp. 556–569.
- [26] J. Bausch, A. W. Senior, F. J. H. Heras, T. Edlich, A. Davies, M. Newman, C. Jones, K. Satzinger, M. Y. Niu, S. Blackwell, G. Holland, D. Kafri, J. Atalaya, C. Gidney, D. Hassabis, S. Boixo, H. Neven, and P. Kohli, “Learning to decode the surface code with a recurrent, transformer-based neural network,” *arXiv preprint arXiv:2310.05900*, 2023.
- [27] O. Higgott, “Pymatching: A python package for decoding quantum codes with minimum-weight perfect matching,” 2021. [Online]. Available: <https://arxiv.org/abs/2105.13082>
- [28] J.-P. Tillich and G. Zémor, “Quantum ldpc codes with positive rate and minimum distance proportional to the square root of the blocklength,” *IEEE Transactions on Information Theory*, vol. 60, no. 2, pp. 1193–1202, 2014.
- [29] A. Leverrier, J.-P. Tillich, and G. Zémor, “Quantum expander codes,” in *2015 IEEE 56th Annual Symposium on Foundations of Computer Science*. IEEE, 2015, pp. 810–824.
- [30] O. Fawzi, A. Grospellier, and A. Leverrier, “Constant overhead quantum fault tolerance with quantum expander codes,” *Communications of the ACM*, vol. 64, no. 1, pp. 106–114, 2021.

- [31] —, “Efficient decoding of random errors for quantum expander codes,” in *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing*, 2018, pp. 521–534.
- [32] P. Panteleev and G. Kalachev, “Quantum ldpc codes with almost linear minimum distance,” *IEEE Transactions on Information Theory*, vol. 68, no. 1, pp. 213–229, 2022.
- [33] —, “Degenerate quantum ldpc codes with good finite length performance,” *Quantum*, vol. 5, p. 585, 2021.
- [34] A. Y. Kitaev, “Fault-tolerant quantum computation by anyons,” *Annals of physics*, vol. 303, no. 1, pp. 2–30, 2003.
- [35] A. deMarti iOlius, P. Fuentes, R. Orús, P. M. Crespo, and J. Etchezarreta Martinez, “Decoding algorithms for surface codes,” *Quantum*, vol. 8, p. 1498, Oct. 2024. [Online]. Available: <http://dx.doi.org/10.22331/q-2024-10-10-1498>
- [36] P. Das, A. Locharla, and C. Jones, “Lilliput: a lightweight low-latency lookup-table decoder for near-term quantum error correction,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 541–553.
- [37] S. Vittal, P. Das, and M. Qureshi, “Astrea: Accurate quantum error-decoding via practical minimum-weight perfect-matching,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–16.
- [38] N. Alavisamani, S. Vittal, R. Ayanzadeh, P. Das, and M. Qureshi, “Pro-match: Extending the reach of real-time quantum error correction with adaptive predecoding,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 818–833.
- [39] S. Liang, J. Xu, Y. Lu, H. M. Chen, B. Yuan, and H. Fan, “Hardware-efficient union-find decoder towards scalable topological quantum codes,” in *2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2026, pp. 77–82.
- [40] X. Fang, M. Wang, Y. Wu, S. Prabhu, D. Tullsen, N. R. Miniskar, F. Mueller, T. Humble, and Y. Ding, “Lightstim: A framework for qec protocol evaluation and prototyping with automated dem construction,” *arXiv preprint arXiv:2604.21472*, 2026.
- [41] A. B. Ziad, J. Xu, and H. Fan, “Greenpeas: Unlocking adaptive quantum error correction with just-in-time decoding hypergraphs,” *arXiv preprint arXiv:2604.16613*, 2026.